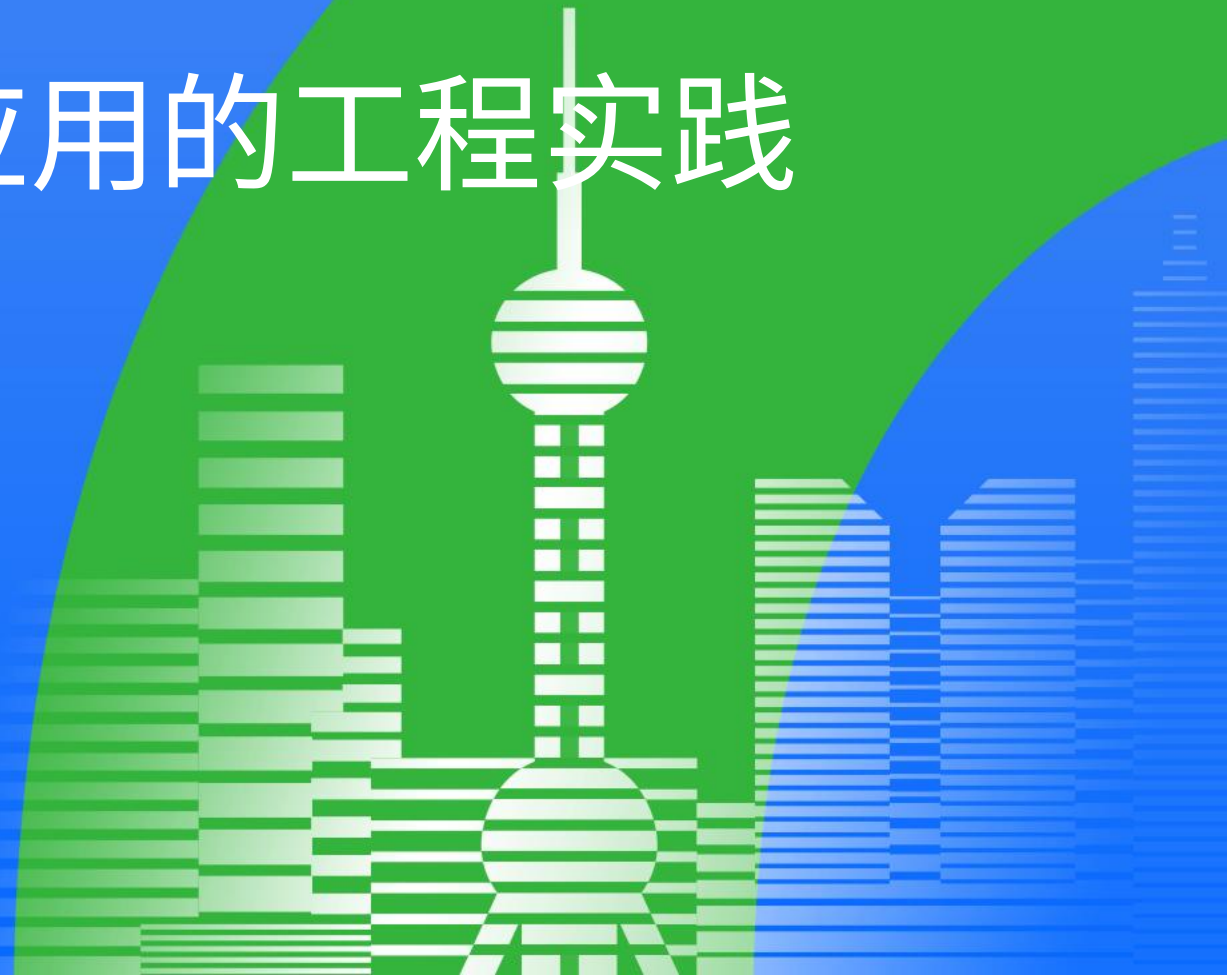
QCon
全球软件开发大会

InfoQ 技术大会

端侧优先，云端兜底
端云协同调度在B端应用的工程实践



李东东

快手电商B端智能经营负责人

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 为什么 B端 场景需要端云协同?

02 端智能体系的设计

03 端智能体系在 B端 落地实践

04 总结及规划

01

为什么B端场景需要端云协同?

从“延迟高、成本贵、数据出域”的三难困境出发，引出“不是二选一，而是如何分工”的核心命题

现状分析

B端AI正在被“体验、成本、合规”三座大山拖住脚步　　用户要的不是对话，是能直接操作的界面

核心数据

20+	定制UI组件
60%	人力投入
3-5天	每个新界面从需求到上线
3个月+	20+组件累积排期

业务场景举例

- 商家说：帮我查近7天订单列表 → 需要列表组件
- 商家说：帮我改商品库存和价格 → 无法执行操作
- 商家说：诊断一下店铺经营问题 → 不支持该技能

用户要的是能填、能提交、能办事的真实界面

工程约束三座大山

体验　纯云端2-5秒，经营操作等不起

成本　高频请求全走云端，成本线性增长

合规　订单/商品/价格出域，隐私压力大

用户真正想要的

商品名称　请输入商品名称

商品价格　请输入价格

商品图片　请选择图片

立即上架　重置

- 可交互
- 可提交
- 真实组件

VS

实际拿到的

```
## 商品上架
- 名称：__
- 价格：__
- 库存：__

[立即上架]
```

- ✗ 静态文本
- ✗ 用户手动复制

这三个约束，纯云端方案有一定难度。那纯端侧呢?

那纯端侧是否可行?

纯端侧看起来能解决延迟与合规 – 数据不出域，响应快，成本低

算力有限

小模型处理不了复杂推理与长文本



设备碎片化

CPU/GPU 差异大，适配成本高

冷启动

模型加载耗时长，首次体验差

所以问题不是选哪边，而是怎么分工?

不是二选一：纯云端和纯端侧各有硬边界

纯云端推理	对比维度	纯端侧推理
能力强，复杂任务效果好	优势	低延迟、低成本、数据不出域
受网络、排队、推理影响，通常 2-5s延迟	延迟	受设备算力和冷启动影响
随调用量线性增长	成本	边际成本接近 0
敏感经营数据需上传	合规	数据可留在本地
弱网、断网不可用；高频轻量请求浪费大模型能力	硬边界	设备碎片化：小模型不适合复杂推理与长文本

行业共识

Apple

- on-device processing + Private Cloud Compute
- 复杂请求进入隐私增强云端

Microsoft

- Copilot+ PC 推动
- 40+ TOPS NPU 成为本地AI基础能力

Chrome

- built-in AI / Gemini Nano
- Web 应用可在桌面端本地执行部分AI任务

我们的分工结论

规则引擎	确定性、高频、可枚举任务
端侧模型	轻量语义理解、分类、参数抽取、简单生成
云端模型	复杂推理、长文本、多轮规划、高质量生成
混合流水线	端侧预处理 + 云端推理 + 端侧后处理

“端云协同” 已是行业共识，但真正的问题在于： 端云协同的架构应该如何设计？业务如何落地？

分享核心脉络

端侧优先，云端兜底
端云协同调度在B端应用的工程实践

本次分享从B端AI落地“纯云端延迟高成本贵、纯端侧算力不足”的两难困境出发，分享我们构建端云协同调度系统的实践：规则引擎短路、端侧优先推理、云端兜底复杂任务，三层决策链动态协同，以及“端侧优先”原则下的取舍与踩坑



端智能体系设计及优化



端智能在B端的落地实践



未来规划及展望

02

端智能体系设计

围绕调度器展开三层决策链设计，涵盖模型选型对比、性能优化与混合流水线架构

端智能体系设计目标与原则

四个目标，需要权衡取舍

成本
越低越好

延迟
越低越好



隐私
数据不出域

效果
越优越好

四个执行原则

01 **规则短路**
确定性任务优先毫秒级返回

02 **端侧优先**
满足条件时优先本地处理

03 **云端为底座**
条件不满足或负责任务由云端承担

04 **混合增效**
端侧与云端分阶段协作

“端侧优先”是方向与原则，云端仍是当前默认路径

端智能体系架构

应用场景

核心经营场景：快手小店，生意通

店铺概览　店铺概览　店铺概览　店铺概览　...

A2UI生成的可交互界面



业务价值

- ✓ 高频操作提效
- ✓ 用户体验一致，智能化
- ✓ 数据隐私
- ✓ 降本增效

业务落地验证

- 试点范围：商品发布，订单发货
- 对比维度：操作时长，完成率，准确率等
- 输出：效果对比报告

交互层

PC web　PC 桌面端　移动端

统一端智能SDK（可插拔）核心层

推理能力

- 推理框架适配层
- 模型管理
- 端侧推理引擎

A2UI能力

- 意图理解
- UI生成引擎
- 组件库 业务适配器

Agent能力

- 推理框架适配层
- 模型管理
- 端侧推理引擎

统一接口层

流式响应　错误处理　日志上报　性能埋点　配置管理

端云协同调度（低延迟 & 隐私保障）

智能调度决策引擎（模型能力，网络质量，敏感数据等）

端侧处理	云端处理	端侧后处理
数据预处理	大模型推理	结果过滤
特征提取	复杂计算	A2UI生成
轻量模型推理	知识检索	本地缓存

智能调度决策引擎（模型能力，网络质量，敏感数据等）

企业级管理平台

- 能力开发 & 配置管理
- 模型管理 & 灰度 & AB实验
- 监控告警 & 链路追踪
- 用量统计 & 成本分析
- 数据安全 & 开发者工具

基础服务层

模型服务	知识服务	Agent服务	数据服务	缓存 & 消息	安全服务
模型仓库	知识库	工具市场	业务数据	Redis	鉴权认证
推理服务	文档解析	工作流	特征数据	Kafka	权限管理
向量检索	向量化	记忆服务	日志数据	MQ	密钥管理

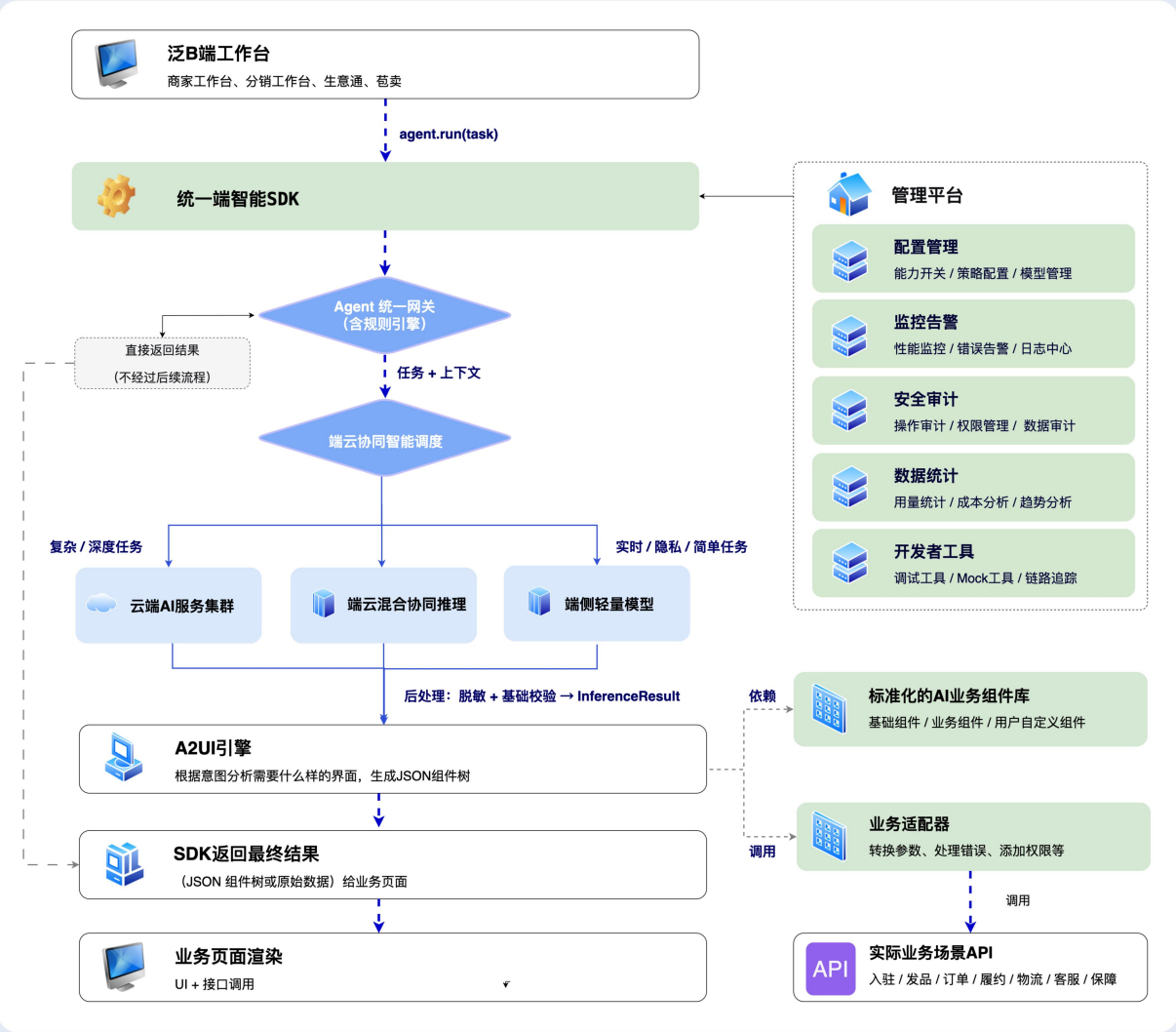
业务落地验证（试点）

高频操作效率提升 ⩾ 30%

基础设施层

公有云/私有云	容器化部署	弹性伸缩	数据存储	网络安全	CDN加速
最小化采集	访问控制	数据脱敏	匿名化采集	审计与追溯	数据分级

端智能体系用户动线



端智能体系动线总览

系统围绕“规则先行、端侧优先、云端兜底”的调度理念，构建了一条从用户请求到业务页面渲染的完整闭环链路。用户从任一B端工作台发起请求后，经由统一端智能SDK接入，经过调度、推理、编译、适配四层处理，最终在业务页面完成可交互界面的渲染输出。

- 统一接入层：多入口统一收口
- 管控与拦截层：规则先行，能不调模型就不调
- 端云协同调度层：三层决策，动态路由
- A2UI编译与适配层：从意图到可交互界面
- 可观测与治理底座：全链路可追踪、可度量

核心：端云调度器 - 第一步不是推理，而是先过两道拦截

B端有大量确定性、可枚举的请求 – 让他们走模型，**既慢又贵**



用户请求

自然语言，操作指令



规则引擎

短路拦截



关键词路由

强制路由



端云调度器

算法决策



执行推理

edge / cloud / pipeline

规则引擎：短路拦截



命中预警 / 远程下发规则 -> **直接返回**

source = ‘rule’ , 完全绕过模型



响应时间接近 0ms



典型案例：固定话术回复，FAQ，已知操作路径

关键词路由：强制路由



命中关键词 -> 强制指定 **excutionMode**

(edge / cloud / pipeline)



优先级高于调度器 算法决策



典型案例：包含 上架 -> 强制走云端 **taskType = product-listing**

设计要点

- 管理平台下发规则，SDK 本地缓存，支持热更新，无需发版
- 规则引擎不依赖推理框架：模型加载失败，核心业务仍可运行
- 规则匹配路径禁止异步操作　保证低延迟的设计约束
- 关键词路由输出“命中哪个路由”，不输出结构化意图



两道拦截的协同关系

拦截层	作用	结果
规则引擎	确定性任务直接返回	source = ‘rule’ , 毫秒级返回
关键词路由	特定场景强制指定执行路径	**优先级高于调度器** 覆盖算法决策

第一步不是让模型“理解”用户说什么 – 而是先问：**这个问题，根本不需要模型**

核心：端云调度器 - 三层递进式决策链



01 规则引擎短路

成本最低　延迟≈ 0

职责	处理确定性、高频、可枚举任务
典型场景	固定话术回复、FAQ、已知路径、简单指令匹配
关键指标	响应时间接近 0ms，完全绕过模型调用

02 端侧推理

低成本　延迟 <100ms

职责	轻量语义理解、意图分类、参数抽取、简单生成
典型场景	FAQ、intent-classify、keyword-filter
关键指标	目标延迟 < 100ms（预热 + 缓存 + 轻量任务）

03 云端推理

能力最强　延迟2-5s

职责	复杂推理、长文本生成、多轮规划、高质量内容生成
典型场景	long-form-generation、复杂经营分析
关键指标	受网络、排队、推理影响，通常 2-5 秒

切换条件　调度器实时评估四个维度

任务复杂度	low → 端侧优先；medium → 混合流水线；High → 云端优先
端侧模型就绪状态	模型是否已下载，已加载，可用
设备性能评分	⩾70 高性能；30-69 中端；<30 低端
网络质量	offline / poor / good

降级路径　每一层都预声明fallback

执行层	触发条件	Fallback路径
规则引擎	规则未命中，或命中但非直接返回	进入关键词路由 → 端云调度
端侧推理	模型未就绪 / 任务复杂 / 推理失败	降级到云端推理
云端推理	超时 / 返回异常 / 网络不可达	降级到端侧 或 规则 或 错误态兜底
错误态兜底	以上全部失败	输出最小可渲染错误态组件

关键设计原则

决策是一次性的
在执行前算一次，不是在执行过程中持续重评估

每个场景都预先声明了fallback
降级路径不是临时决定的

“端侧优先”是设计原则
但云端仍是当前默认路径

关键：调度方案设计 - 八场景决策矩阵



每个场景都预声明了 *fallback* – *降级路径不是临时决定的*

端侧模型选型对比

对比维度	WebLLM	Ollama	ONNX Runtime	WebGPU / 原生 NPU
推理性能 首token延迟，吞吐量	首token延迟 中等 吞吐量 中等	首token延迟 较低 吞吐量 较高	首token延迟 较低 吞吐量 较高	首token延迟 较低 吞吐量 较高
内存占用 运行时内存、模型加载	运行时内存 中等 常驻内存 中等	运行时内存 较高 常驻内存 较高	运行时内存 较低 常驻内存 中等	运行时内存 较低 常驻内存 较低
框架兼容性 跨平台支持，平台适配成本	跨平台支持 强 适配成本 低	跨平台支持 中等 适配成本 中等	跨平台支持 强 适配成本 中等	跨平台支持 中等 适配成本 较高
模型格式 模型文件类型、格式	GGUF ONNX	GGUF 其他	ONNX 其他	ONNX 其他

方案对比

方案	状态	选型判断
WebLLM	已落地	通过bridge在本地执行，浏览器及桌面端通用
Ollama	已落地	桌面端主要验证路径，模型生态成熟
ONNX Runtime	接口已预留 尚未实现	适配器为stub, 需引入onxxruntime-web后激活
WebGPU	选型评估中	设备与浏览器能力差异明显，生态碎片化

选型决策逻辑

01 优先保证可用性

优先选用 webllm , ollama 作为主力

02 接口层做抽象

推理引擎统一接口，支持运行时切换

03 新后端渐进式接入

先预留接口，条件成熟后激活

必要说明

“接口抽象好了”与“后端都可用了”是两件不同的事

核心：模型无损热更新设计

双版本共存，会话绑定，优雅排干与卸载



实际效果

- **旧模型服务已有会话**：上下文不丢、状态不丢、用户无感知
- **新模型服务新会话**：新用户第一时间用上新能力
- **云端失败时自动降级**：已有会话不受影响

用户不关心模型什么时候升级的，他只知道上次没有填完的表单，这次打开还在

端云协同的终极形态

混合流水线：端侧模型不只是云端的替代，也可以是云端的前后处理器

端侧预处理

- 特征提取、意图识别、脱敏、上下文裁剪
- 目标：减少token量，敏感数据不出本地
- 执行位置：端侧

云端推理

- 复杂推理、长文本生成、高质量内容生成
- 目标：发挥大模型能力上限
- 执行位置：云端

端侧后处理

- Schema教研，字段回填，A2Ui渲染
- 目标：结构收敛，确保输出内容合法可渲染
- 执行位置：端侧

内置模版示例

模版	执行步骤	端侧价值
production-description	端侧特征提取 -> 云端生成 -> 端侧后处理	原始商品数据不出本地
customer-service	端侧意图分类 -> 云端生成回复	减少云端prompt复杂度
summary	端侧关键句提取 -> 云端摘要	减少上传token量，降低延迟

三级降级阶梯

1. → 返回固定模版或确定性工具路径
2. 第2级：云端兜底 → 端侧不适合或不可用时切云端
3. 第3级：错误态兜底 → 输出最小可渲染错误态，保证不白屏

真实踩坑：如果云端失败后，不再fallback到端侧继续拼接，否则两个模型会混合输出

降级不是异常分支，而是**主流程的一部分**

03

端智能体系在B端落地实践

分享真实链路建设进展、可观测性现状与缺口，以及“端侧优先”原则下的踩坑经验

商家经营落地架构总览

端侧实时感知 / 执行 × 云端深度推理 × 端云协同 = 让商家的每一步经营都有“智能副驾”

业务问题

- 数据多，不知道看什么
- 建议发了，不知道有没有用
- 首页更多的是信息展示，不是行动指引或无法执行
- 纯云端链路成本高、响应慢、隐私压力大

端侧实时感知	云端深度推理	端云协同调度
行为采集	诊断生成	请求分流
时机判断	效果归因	实时触发
本地识别	分型优化	闭环学习
结果呈现	建议编排	状态同步

端侧负责实时性，云端负责复杂性，协同负责最优路径

业务结果

- 打开平台就有答案
- 做了就能看到效果
- 首页知道今天需要做什么
- 给了建议后，在必要时可以自动执行

1. 端智能实时诊断

2. 端智能体系在业务闭环

3. 桌面端升级

产品 × 端侧 × 服务端 × 算法 × 数据协同

落地过程中的四大难点与解法

最大的复杂度不在模型，而在状态

痛点

设备碎片化	CPU/GPU/NPU、内存、系统版本差异大，端侧能力参差不齐
可观测性弱	决策时不知道网络下一秒是否抖动 模型是否能加载成功
模型/加载性能	模型首次加载耗时数秒，用户等不起
模型能力无法支持业务复杂度	当前在B端业务中，对准确率及稳定性要求比较高，但模型在某些时候还是无法支持

状态维度

- 设备
- 网络
- 模型
- 执行

应对策略

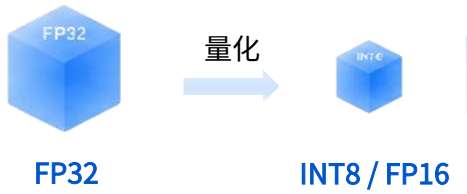
能力探测	启动时采集设备、网络、模型状态，生成设备分
配置化降级	不同租户，设备，场景走不同策略，低端设备禁用端模型
预加载	空闲预加载，按租户与场景配置
模型热更新	双版本共存，新请求走新模型，旧请求完成后卸载
完整性校验	模型包，组件包做hash校验，失败后拒绝加载
限制使用场景	并不是所有的能力都让调用AI，要因地制宜的使用

端侧优先，不能以牺牲用户设备体验为代价

难点1：模型性能/加载优化

01 第一板斧：量化

降低精度，减小体积，提升速度



- 降低模型精度，减少模型体积
- 减少推理时的计算量，提升推理速度
- 适合端侧，精度损失在可接受范围

体积更小 ↓ 计算更少 ↓ 速度更快 ↑

02 第二板斧：预加载

提前准备，首推秒开



- 登陆后或空闲时预加载
- 按租户与场景配置预加载策略
- 低端设备禁用预加载

可配置 ⊗ 可关闭 因机制宜

03 第三板斧：缓存命中

常驻内存，复用无感



- 模型加载后常驻内存，后续请求直接使用
- 缓存命中时模型加载 < 200ms
- 配合预加载策略，将冷启动转化为热启动

常驻内存 < 200ms 热启动

效果数据（目标值）

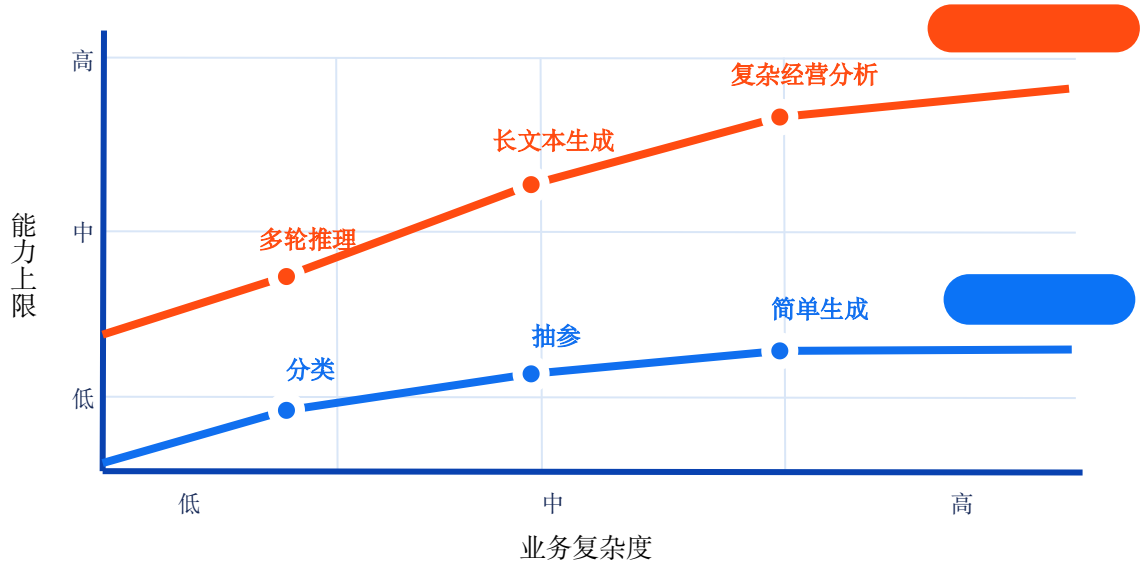
预加载策略的核心原则 – 不该让端侧优化的成本，转嫁到用户每次操作的等待上

难点2：模型能力 vs 业务复杂度

现象

- 端侧小模型
- 云端大模型
- 结构化输出

能力上限　业务复杂度



推理后端现状

方案	状态
WebLLM	已落地
Ollama	已落地
ONNX Runtime	接口已预留，尚未实现
WebGPU / NPU	选型评估中

设备现实

OOM

应对策略

01 规则优先

02 任务拆解

03 UIPlan中间层

04 归一化　图校验

不要让端模型承担它 不擅长 的任务

难点3：全链路监控提升可观测性

监控是基础，可观测性才是目标

已具备

- SDK调用量，成功率，耗时
- 推理来源耗时，成功率
- 模型加载耗时
- A2UI生成耗时、成功率、兜底率
- 按租户、平台、环境等维度拆分



尚缺

- ✗ 统一的运行事件模型
  无法串联一次需求的完整链路
- ✗ 一次请求的完整可重建轨迹
  当前需拼多分日重推断
- ✗ 离线回放与自动化回归评测
  无法做回测验证

目标事件流

01	02	02	04	04	06	07	08	09	10
请求接收	规则命中	路由选择	推理开始/结束	UI 呈现	用户确认	工具调用	降级触发	响应渲染	反馈上报
记录请求上下文	匹配结果与动作	选择执行与结果	记录耗时与结果	组件生成与渲染	点击 / 修改 / 提交	外部工具执行	触发原因与目标	最终输出内容	成功 / 失败 / 评分

能算P95还不够，当用户说“那次生成不对”时，我们需要能完整回放那次执行

落地收益：工程效能、端智能架构、业务体验

工程效能提升	端智能架构提升	业务体验改善
组件开发成本下降 XX% 标准化组件覆盖7大分类，UI需求无需从零开发，UIPlan自动编排即可生成可用界面	**规则命中率达到 XX%** 确定性高频请求完全绕过模型，毫秒级返回，大幅降低不必要的云端调用	**规则命中延迟 < XXms** 目标值，匹配路径禁止异步操作，确定性任务实现毫秒级响应
Prompt维护成本下降 XX% 从1173行手写Schema常量降至约200行自动派生，消除手工维护与真实渲染层之间的漂移风险	**端侧推理占比提升至 XX%** 低复杂度、设备达标、模型就绪时优先走端侧，降低云端成本与数据出域风险	**端侧推理延迟 < XXms** 目标值，模型预热 + 缓存命中 + 轻量任务组合下，弱网/离线场景仍可正常使用
模型更新周期缩短 XX% 模型包独立于宿主，后台静默升级，用户无感知，迭代周期从天级缩短至小时级	**云端调用成本下降 XX%** 高频任务端侧消化，云端只处理复杂推理，调用成本随用量线性增长的压力得到缓解	**A2UI端到端延迟 < XXms** 目标值，含1-2次推理串行 + 编译 + 序列化，用户感知的完整链路延迟控制在合理范围

收益不只是快和省　　更关键的是，我们终于知道怎么把AI‘放’进B端工作流里了

04

总结及规划

从可运行走向可观测、可评测、可复用，分享统一事件模型、离线回放与开源计划

经验总结

经验一

端云协同不是模型部署位置的选择题，而是业务执行路径的调度题

经验二

规则解决确定性，端侧解决实时性，云端解决复杂性

规则 确定性 → 端侧 实时性 → 云端 复杂性

一个原则



B端AI的终点不是回答

而是 可确认、可执行、可审计 的业务动作

- 可确认
- 可执行
- 可审计

工程规范

01 规则能解决的，不进模型

02 端侧能稳定解决的，不上云

03 复杂任务上云，但需安全

04 模型不强制约束结构

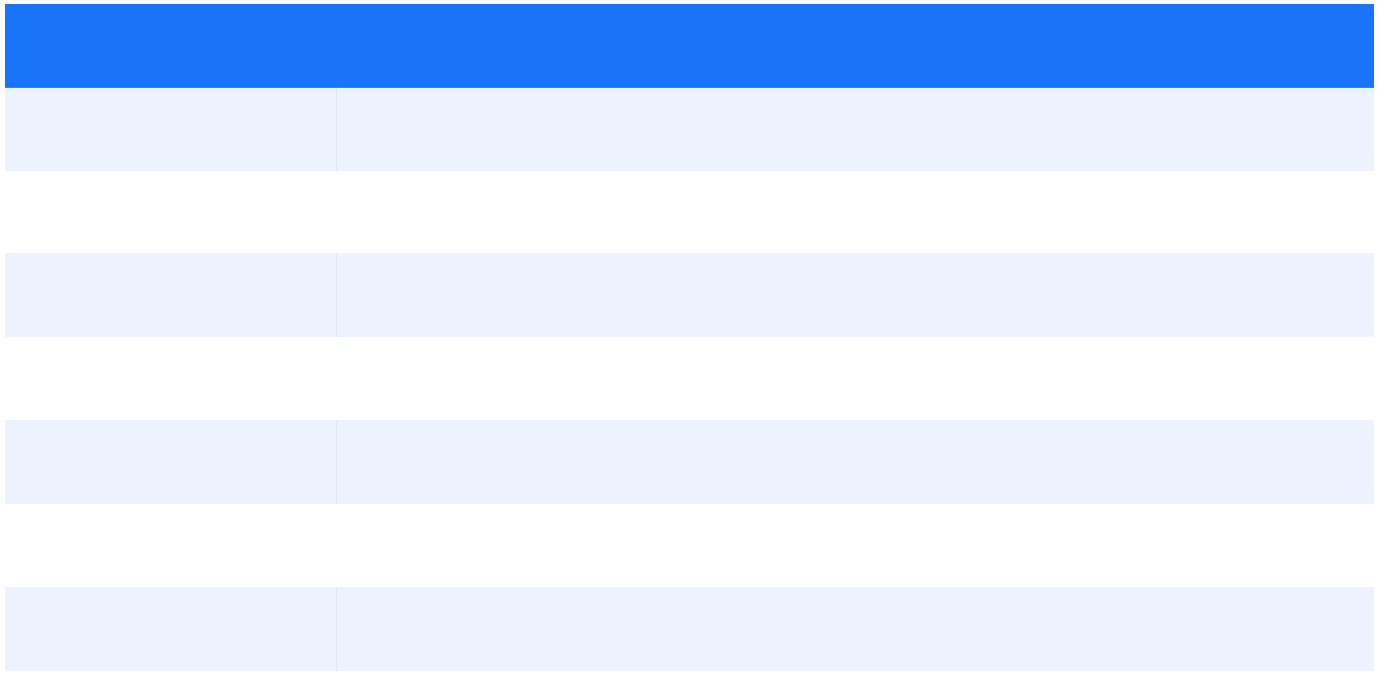
05 失败必须可降级：规则、云端、兜底

06 所有路径都要可观测、回放、治理

规划：从可运行，走向可观测、可评测、可复用

端智能系统一旦进入生产，最重要的不是继续堆能力，而是让每一次运行 **可解释、可回放、可度量**

下一阶段重点



开源定位

不是

端侧模型SDK

推理框架已有成熟框架

而是

Local-first Business Agent Runtime

面向业务工作流的本地优先、配置驱动的Runtime

核心差异

把本地模型、规则、A2UI与业务工具组织成可治理、可执行、可回放的业务工作流

Local-first Business Agent Runtime： 把本地模型、规则、A2UI与工具组织成 **可治理、可执行、可回放** 的业务工作流

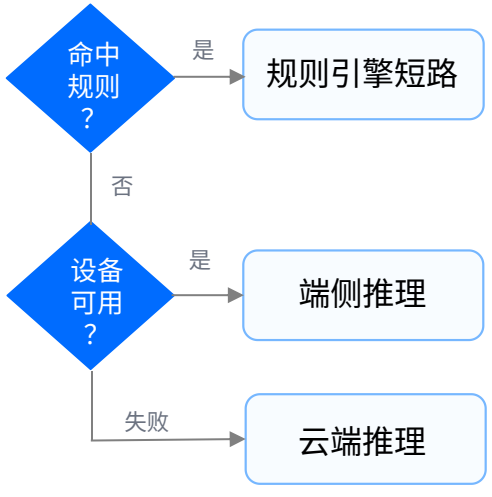
规划：两个关键设计取舍，我们还在探索的方向

01 前沿亮点一 – 三层递进式调度决策体系

规则引擎短路 → 端侧推理 → 云端推理

调度器综合评估

- 任务复杂度
- 设备性能
- 网路质量



- 云端失败时自动降级
- 每个场景预声明fallback
- 降级时主流程的一部分

02 前沿亮点二 – 乐观路由 + 执行中重评估

先按最优路径执行 → 执行中持续监测 → 实时降级

请求输入 → 按最优路径执行 → 执行中监测
- 正常完成 → 输出结果
- 出现问题 & 实时降级 → 输出结果

创 **独创性：边执行边决策**

流失场景真实教训

云端失败后不建议fallback到端侧拼接

调度要 可解释、可降级、更要贴近真实场景

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

